CloudChain: Attack-Path Analysis for AWS with Read-Only Validation Against the Live Account

Parth Joshi
Symbiosis Skills and Professional University
Pune, India

Hrishikesh Kanapuram
Symbiosis Skills and Professional University
Pune, India

Aaryaman Bhatnagar
Symbiosis Skills and Professional University
Pune, India

Saahithi Mukala
Symbiosis Skills and Professional University
Pune, India

Dr. Sunil Kale
Symbiosis Skills and Professional University
Pune, India

***Abstract*—Cloud security posture management (CSPM) tools usually report AWS misconfigurations as a flat list, one check at a time. Breaches tend to come from chains of weaknesses instead, and the tools that do model chains derive them from configuration and present them as fact. We describe CloudChain, an open-source CSPM tool that builds a graph of AWS resources and identities, searches it for routes to administrator access, and then re-checks every hop of every route against the live account using read-only API calls. For permission hops it asks AWS's own policy simulator, which applies explicit denies, permissions boundaries and service control policies (SCPs) that static analysis does not model. Each hop comes back CONFIRMED, REFUTED or UNVERIFIABLE, together with the API calls behind the verdict. We evaluated CloudChain on a real AWS account governed by an organisation SCP, using nine IAM test identities with known ground truth. CloudChain and Prowler both flagged all six escalation-capable identities and none of the three negative controls. Validation then showed that four of the seven routes CloudChain derived (57%) could not be walked because the SCP denied an action they depended on, a distinction Prowler's output does not make. After we removed the escalation permission from two identities, re-validating the stored scan marked exactly those two routes REFUTED. On synthetic organisations the analysis took 2.6 s for 10,000 identities, and a hop limit kept path search bounded on dense graphs where unbounded enumeration exceeded 60 s at 10 identities.**

***Index Terms*—cloud security, AWS IAM, attack graphs, privilege escalation, security posture management, policy validation**

I. Introduction

Free CSPM tools such as Prowler [1] and ScoutSuite [2] run hundreds of independent checks against an AWS account and report each failure with a fixed severity. A public bucket, a user without MFA and a policy that grants `iam:PassRole` appear as separate rows. An attacker sees them differently: the bucket exposes a key, the key belongs to the user, and the user can pass an administrator role into a new Lambda function. The risk is the chain, and a flat list hides it.

Graph-based tools address this. PMapper [3] and awspx [4] model IAM principals and the edges between them, and commercial products present attack paths on a dashboard. These paths are still outputs of a model built from configuration. A path can look complete and still be blocked by an SCP, a permissions boundary or an explicit deny that the analyser never read. Security teams that have seen enough blocked paths learn to distrust the whole category of finding.

CloudChain treats a reported path as a claim to be tested. This paper makes four contributions:

1) A resource graph for AWS that separates same-account `iam:PassRole` escalation from cross-account `sts:AssumeRole` movement, so that no reported chain crosses an account boundary through a mechanism AWS does not allow (Section III).
2) A read-only validation engine that re-checks each hop of each path against the live account and consults `iam:SimulatePrincipalPolicy` [13] so that SCPs, boundaries and explicit denies can overturn a statically derived hop (Section IV).
3) An explainable posture score and a per-finding risk score whose arithmetic is published alongside every number, with a weight-sensitivity analysis of both (Sections V and VII-F).
4) An evaluation on a real SCP-governed account against Prowler, a stale-path experiment, and scalability measurements on synthetic organisations (Section VII).

The source code, the test suite and every experiment script are public [18].

II. Related Work

Attack graphs. Sheyner et al. [9] generated attack graphs for networks with model checking, and MulVAL [10] and its successor [11] made generation scale by expressing exploit rules in Datalog. CloudChain applies the same idea to cloud identities, where edges come from IAM grants and trust policies instead of host vulnerabilities.

Cloud scanners. Prowler [1] and ScoutSuite [2] are check-based. Prowler includes checks for policies that allow privilege

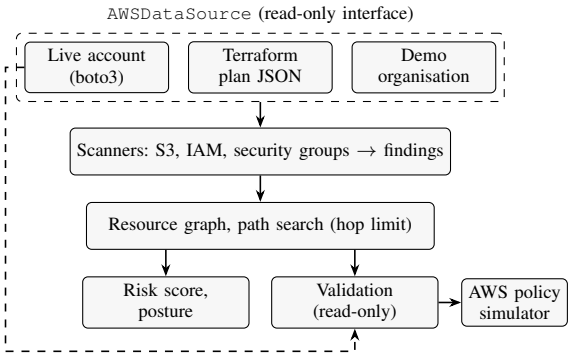


Fig. 1. CloudChain pipeline. All analysis runs against one read-only data-source interface; validation returns to the live account.

escalation, but it reports the policy, not the route it opens or whether that route is usable under the account's SCPs. Cloudsplaining [6] reports risky actions in IAM policies, also per policy.

IAM graphs. PMapper [3] builds a graph of IAM principals and answers who can reach whom, and awspx [4] visualises similar relationships. Cartography [5] loads cloud assets into a graph database for querying. The escalation primitives these tools and CloudChain look for were catalogued by Gietzen [8]. BloodHound [7] popularised attack-path analysis for Active Directory.

Policy reasoning. Zelkova [12] encodes AWS policies as SMT formulas and decides properties of them exactly. CloudChain does not reason about policy semantics itself beyond wildcard matching. It delegates the final question to AWS's policy simulator [13], which implements the documented evaluation logic [14] including SCPs [15].

None of the open tools above returns to the account to test whether a derived path is still walkable, and none records the evidence for a path in a form a reviewer can reproduce by hand. That is the gap CloudChain addresses.

III. System Design

Fig. 1 shows the pipeline. Every scanner is written against a single interface, `AWSDataSource`, which exposes only read operations. Three implementations exist: a live account through boto3, a Terraform plan, and a seeded three-account demo organisation. Because the scanners never know which source they read, the analysis that gates a pull request is the same code that runs after deployment.

A. Findings

Scanners emit findings for S3 (public access, credential-shaped object keys in public buckets, encryption, versioning), IAM (missing MFA, stale keys, administrator-equivalent policies, escalation primitives, `iam:PassRole` combined with a compute-creation action, cross-account trust) and EC2 security groups open to the internet. Policies are read in full: attached AWS-managed, attached customer-managed and inline. Action patterns are matched with IAM's wildcard semantics, so a grant of `iam:*` or `iam:Pass*` covers `iam:PassRole`. An identity holding `iam:*` on all resources is treated as administrator-equivalent, because it can attach `AdministratorAccess` to itself in one call.

B. Graph model

Nodes are resources namespaced by account (`111111111111/iam_user:svc`), and each account has its own administrator sink. Edges carry one of the following relations:

- `leaks_credentials_for`: a public bucket exposes an access key belonging to a user;
- `can_pass_role_to`: an identity holds `iam:PassRole` on a role and an action that creates compute (Lambda, EC2) to run as it;
- `can_assume_cross_account`: a role's trust policy names a principal in another account;
- `can_escalate_to_admin`: an identity holds an IAM write permission that lets it grant itself administrator access;
- `grants_admin_access`: an identity holds an administrator-equivalent policy.

`iam:PassRole` is same-account by construction in AWS, so a `can_pass_role_to` edge never crosses an account boundary. Crossing requires `sts:AssumeRole` against a trust policy. Keeping the two separate prevents chains that look plausible and cannot be walked; a unit test asserts it.

C. Paths

An *attack path* starts at an entry point an unauthenticated attacker can reach (a public, sensitive resource) and ends at an administrator sink. An *escalation route* starts at an ordinary identity and ends at a sink; it matters only once that identity is compromised. The two are reported and scored separately so that a latent primitive never scores as badly as an open door. Paths are enumerated with simple-path search bounded by a hop limit (default six), because the number of simple paths grows factorially on densely connected graphs (Section VII-E).

D. Opt-in credential correlation

In the demo organisation the owner of a leaked key is known. On a real account, a key name such as `credentials.csv` does not say whose key it is. With an explicit opt-in flag, CloudChain reads the first 64 KB of credential-shaped objects in public buckets, extracts access key IDs with a regular expression, and matches them against `iam:ListAccessKeys`. Secret keys are never extracted, object bytes are discarded after matching, and only masked IDs reach the report. Keys owned by another account are identified with `sts:GetAccessKeyInfo` and reported, not guessed at.

IV. Read-Only Path Validation

For each hop of a path, the validation engine runs a verifier specific to the edge relation, records every API call it made, and returns one of three verdicts:

- CONFIRMED: the API responses show the hop's precondition holds now;
- REFUTED: a response contradicts it, so the path is stale or blocked;
- UNVERIFIABLE: the responses were not enough to decide, for example because a read was denied.

A path is REFUTED if any hop is, CONFIRMED only if every hop is, and UNVERIFIABLE otherwise. Missing information never produces CONFIRMED.

A. Verifiers

The S3 hop re-reads the bucket's public access block, ACL and policy status, re-lists object keys, and checks that the leaked user still has an active access key. The PassRole hop re-reads the identity's policies and checks that `iam:PassRole` still covers the target role and that a compute-creation action is still granted. The self-escalation hop checks that at least one escalation primitive is still granted. The administrator hop checks for an administrator-equivalent statement. The cross-account hop re-reads the target role's trust policy in the account that owns the role.

B. Second opinion from AWS

Static analysis in CloudChain does not model explicit denies, permissions boundaries, SCPs or condition keys. After a static verifier confirms a permission hop, CloudChain calls `iam:SimulatePrincipalPolicy` [13] for the actions the hop needs. If the simulator denies them, the hop becomes REFUTED and the reason names the layer that blocked it (SCP, boundary or explicit deny). If a denial depends on a condition key the simulator has no value for, the hop becomes UNVERIFIABLE, not REFUTED. If the scanning identity may not call the simulator, the static verdict stands and the evidence says so.

The simulator also settles a case static analysis cannot. When an identity's own policies come back empty, the identity may have no permissions, the read may have been denied, or the grant may come from an IAM group that CloudChain's static reader does not resolve. The validator asks the simulator and reports REFUTED, CONFIRMED with a note about the indirect grant, or UNVERIFIABLE when no simulator is available.

C. Safety

Validation never performs an escalation: no role is assumed, no function created and no policy attached. Verifying that an identity *can* pass a role into Lambda means confirming the grant, not using it. A CONFIRMED verdict therefore means every precondition holds now, which is weaker than a demonstrated exploit and stronger than configuration inference. The data-source methods validation may call are listed in an allowlist, and a test wraps the data source in a tripwire that fails the build if validation calls anything else, including the opt-in object read. Every recorded call carries its `aws` CLI equivalent so a reviewer can reproduce the verdict by hand.

V. Scoring

Each finding receives a contextual score

$$s = w_{sev} \cdot m_{net} \cdot m_{sens} \cdot m_{path}, \tag{1}$$

where $w_{sev} \in \{1, 3, 6, 10\}$ for LOW to CRITICAL, $m_{net} = 1.5$ if the resource is internet-facing, $m_{sens} = 1.5$ if it holds credentials or sensitive data, and $m_{path} = 2.5$ if it lies on an internet-reachable attack path; each multiplier is 1 otherwise. The interface shows the chain of multipliers for every finding.

The account posture score starts at 100 and subtracts four dimension deductions: exposure (up to 25 points, internet-facing findings weighted by severity), privilege (up to 25, identity weaknesses weighted by how far they advance an attacker), reachability (up to 30, from the existence and shape of routes to administrator access) and blast radius (up to 20, the fraction of the graph reachable from exposed entry points). Each dimension publishes its formula and the findings behind it, and a test asserts that the headline number always equals 100 minus the published deductions. A confirmed internet-reachable route to administrator access caps the grade at F.

VI. Implementation

CloudChain is written in Python with FastAPI, networkx [16] for the graph and boto3 for AWS access, with a React front end and a command-line interface that runs scans, validations and Terraform plan checks without the server. The test suite has 226 tests covering scanners, graph construction, path search, scoring, the posture contract, validation verdicts and safety, credential handling and the plan gate. Continuous integration runs the suite on Python 3.10 and 3.13, runs a demo scan on a clean machine, and checks that the plan gate blocks a known-dangerous Terraform plan. Submitted credentials are held in memory for one scan, never persisted, and never returned to the browser; a test asserts the last property.

VII. Evaluation

We asked four questions. Does CloudChain find the escalation-capable identities a mature scanner finds (RQ1)? Does validation separate routes that can be walked from routes that cannot (RQ2)? Does re-validation notice when a route is fixed (RQ3)? How do run time and the scores behave as inputs change (RQ4)?

A. Setup

We used a real AWS account that belongs to an AWS Organization. The organisation applies an SCP that explicitly denies, among others, `lambda:CreateFunction`, `s3:CreateBucket`, `ec2:RunInstances` and `ec2:DescribeSecurityGroups`, which we confirmed with the policy simulator before the experiment. The SCP rules out S3, EC2 and Lambda resources, so the evaluation covers IAM only. Scans used a dedicated user with the AWS-managed `SecurityAudit` policy plus `iam:SimulatePrincipalPolicy`.

A script in the repository creates nine test users, each with a single inline policy, and two roles: one with

TABLE I
TESTBED IDENTITIES AND OUTCOMES. P = PROWLER ESCALATION FINDING; S = CLOUDCHAIN ROUTES FROM STATIC ANALYSIS; V = CLOUDCHAIN VALIDATION VERDICT († BLOCKED BY THE ORGANISATION'S SCP).

User	Inline policy grants	P	S	V
attach	`iam:AttachUserPolicy`	yes	1	CONFIRMED
policyver	`iam:CreatePolicyVersion`	yes	1	CONFIRMED
putpolicy	`iam:PutUserPolicy`	yes	1	CONFIRMED
addgroup	`iam:AddUserToGroup`	yes	1	REFUTED†
passrole-lambda	PassRole (admin) + Lambda	yes	1	REFUTED†
wildcard-pass	`iam:Pass*`, `lambda:Create*`	yes	2	REFUTED†
passrole-scoped	PassRole (harmless) + Lambda	no	0	n/a
readonly	`iam:Get*`, `iam:List*`	no	0	n/a
s3only	`s3:*`	no	0	n/a

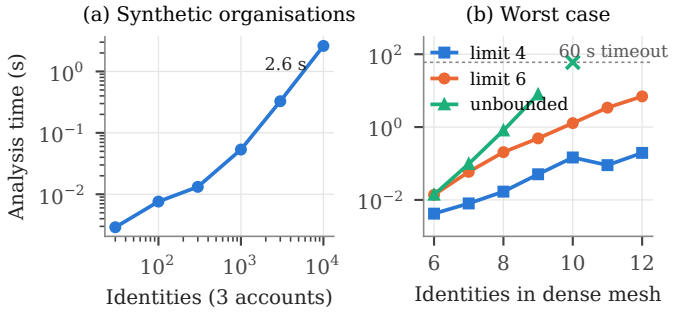


Fig. 2. (a) End-to-end analysis time on synthetic organisations, hop limit 6. (b) Path search on a fully connected trust mesh; the unbounded search timed out at 10 identities.

`AdministratorAccess` and one with no permissions, both trusting Lambda. Table I lists the users. Six hold an escalation capability; three are negative controls. Four management roles created by the organisation were present in the account and were marked as known-benign through CloudChain's suppression setting, which downgrades and annotates findings instead of hiding them.

B. RQ1: Detection compared with Prowler

On the empty account, before the testbed existed, CloudChain reported 8 findings and no paths. Four were on the organisation's management roles and were suppressed as above; the rest (an administrator user, two users without MFA and a missing password policy) were correct.

With the testbed in place, CloudChain reported 7 escalation routes from the 6 escalation-capable users. The wildcard user produced two routes, one to the testbed's administrator role and one to an organisation role that also holds administrator access. Prowler 5 with its IAM checks flagged the same 6 users through its `iam_inline_policy_allows_privilege_escalation` check. Neither tool reported an escalation for the three negative controls (Table I). The user allowed to pass only the harmless role received a CRITICAL escalation finding from CloudChain's IAM scanner, but the graph produced no route for it, because the role it can pass is not administrator-equivalent.

Prowler failed 53 checks in total and took 1 min 45 s; CloudChain reported 24 findings. Raw finding counts are not directly comparable, because Prowler includes checks CloudChain does not implement (for example, confused-deputy protection on service roles).

C. RQ2: Validation under an SCP

Validation confirmed 3 of the 7 routes and refuted 4. Three refutations came from the SCP's denial of the Lambda actions the PassRole routes need. The fourth, `iam:AddUserToGroup`, was denied by the same SCP; we had not known this before the run, and the simulator reported it. In each case the static verifier had confirmed the grant, and the simulator overturned it with the reason "a service control policy, an explicit Deny". A configuration-only analysis would have reported all 7 routes as open. Prowler reported all 6 users at the same HIGH severity and did not indicate that 4 of them could not escalate in this account.

D. RQ3: Detecting stale paths

We then removed the escalation permission from two users (attach and putpolicy) and re-ran validation on the stored scan without rescanning. Both routes changed from CONFIRMED to REFUTED. The policyver route, left untouched as a control, stayed CONFIRMED, and the four SCP-blocked routes stayed REFUTED.

The first attempt at this experiment exposed a defect: with the users' only policy deleted, the validator could not tell an identity with no policies from a read it was not allowed to make, and returned UNVERIFIABLE. We changed the validator to settle that case with the policy simulator (Section IV), added tests for it, and re-ran the same validation, which produced the results above.

E. RQ4a: Scalability

We generated seeded three-account organisations with 30 to 10,000 identities and a fixed mix of administrators, escalation primitives, PassRole grants, cross-account trust and leaking public buckets, and timed the full offline pipeline (median of three runs; AWS API latency excluded). Fig. 2(a) and Table II show the results. At 10,000 identities the analysis took 2.6 s and the longest route had five hops, so the hop limit of six never cut a route and made no measurable difference to run time.

The limit matters on dense graphs. Fig. 2(b) shows a worst case in which every identity can reach every other and one can escalate. Unbounded enumeration found 109,601 routes in 7.9 s at 9 identities and exceeded 60 s at 10. With a limit of six, 12 identities took 7.0 s.

The more pressing limit at scale is the number of routes, not time: 43,752 at 10,000 identities, mostly from wildcard PassRole grants that reach every administrator role. No analyst can read that many, which argues for grouping routes by shared hops.

TABLE II
SYNTHETIC ORGANISATIONS (HOP LIMIT 6, MEDIAN OF 3 RUNS)

Identities	Findings	Edges	Attack	Escalation	Time (s)
30	29	12	0	3	0.003
300	179	101	1	69	0.013
1,000	617	436	0	293	0.054
3,000	1,931	3,540	83	3,125	0.33
10,000	6,394	44,224	860	42,892	2.6

TABLE III
WEIGHT SENSITIVITY, WORST CASE OVER FACTORS 0.5, 0.75, 1.25, 1.5

Weight	$\tau_{\min}$	Top-10	$\lvert\Delta\rvert$ posture	Grade changed
Internet-facing multiplier	0.84	0.76	0	0%
Sensitive multiplier	0.95	0.84	0	0%
Attack-path multiplier	0.86	0.93	0	0%
CRITICAL base weight	0.79	0.55	0	0%
HIGH base weight	0.92	0.74	0	0%
Exposure weight	1.00	1.00	11.2	36%
Privilege weight	1.00	1.00	12.6	36%
Reachability weight	1.00	1.00	9.9	36%
Blast-radius weight	1.00	1.00	0.6	9%
Escalation-only floor	1.00	1.00	2.5	27%

F. RQ4b: Sensitivity to the weights

Every weight in both scores was chosen by hand, so we measured how much they drive the output. We scaled each weight by 0.5, 0.75, 1.25 and 1.5 in turn and compared the result with the baseline on the demo organisation and ten synthetic organisations of 300 identities. For finding scores we report Kendall's τ [17] between baseline and perturbed scores and the share of the baseline top ten that remained in the top ten. For the posture score we report the mean absolute change and the share of organisations whose letter grade changed. Table III gives the worst case over the four factors.

Finding rankings were stable under the multipliers: mean τ stayed at or above 0.95 and the lowest single-organisation τ was 0.84. The largest change came from halving the CRITICAL base weight, which places CRITICAL below HIGH ($\tau_{\min} = 0.79$, top-ten overlap 0.55); that perturbation inverts the severity scale rather than tuning it. The posture score was more sensitive. Scaling the exposure, privilege or reachability weight by 0.5 or 1.5 moved the score by 10 to 13 points on average and changed the grade in up to 36% of organisations. The score's decomposition makes that dependence visible, but the absolute value should be read as relative to the chosen weights until they are calibrated against incident data.

VIII. LIMITATIONS

The real-account evaluation covers IAM only, because the organisation's SCP denies the S3, EC2 and Lambda actions needed to build other scenarios. The internet-to-administrator chain through a leaking bucket is exercised by unit tests and the demo organisation but not on a live account. The testbed has nine identities that we designed, so it shows that the mechanisms work, not how often they matter in production estates.

Real-mode scans cover a single account. The graph model is organisation-wide and the demo exercises three accounts, but scanning a live organisation needs an `sts:AssumeRole` fan-out from a management-account role that CloudChain does not perform.

The static reader does not resolve permissions granted through IAM groups, resource-based policies other than bucket and trust policies, or condition keys. The simulator compensates during validation, but only when the scanning identity is allowed to call it, and the simulator itself does not evaluate resource-based policies for roles [13]. The `iam:AddUserToGroup` edge is drawn without checking that a more privileged group exists, which over-approximates.

The posture weights are not calibrated against incident data, and the sensitivity analysis shows the letter grade depends on them. Route counts grow quickly with wildcard PassRole grants.

IX. CONCLUSION

CloudChain derives attack paths from AWS configuration and then tests them against the account with read-only calls and AWS's policy simulator. On an SCP-governed account it matched Prowler's detection of escalation-capable identities and went further in one respect that matters to an analyst: it showed that four of seven derived routes could not be walked, and it recognised two routes as fixed without a rescan. Future work is a live multi-account scan through role assumption, grouping of routes that share hops, resolution of group and resource policies in the static reader, broader service coverage on an account without restrictive SCPs, and calibration of the posture weights.

REFERENCES

[1] Prowler, "Prowler: open source cloud security platform," GitHub repository. [Online]. Available: https://github.com/prowler-cloud/prowler

[2] NCC Group, "ScoutSuite: multi-cloud security auditing tool," GitHub repository. [Online]. Available: https://github.com/nccgroup/ScoutSuite

[3] NCC Group, "Principal Mapper (PMapper)," GitHub repository. [Online]. Available: https://github.com/nccgroup/PMapper

[4] WithSecure Labs, "awspx: a graph-based tool for visualizing effective access and resource relationships in AWS," GitHub repository. [Online]. Available: https://github.com/WithSecureLabs/awspx

[5] Cartography project, "Cartography," GitHub repository. [Online]. Available: https://github.com/cartography-cncf/cartography

[6] Salesforce, "Cloudsplaining: AWS IAM security assessment tool," GitHub repository. [Online]. Available: https://github.com/salesforce/cloudsplaining

[7] SpecterOps, "BloodHound," GitHub repository. [Online]. Available: https://github.com/SpecterOps/BloodHound

[8] S. Gietzen, "AWS IAM privilege escalation: methods and mitigation," Rhino Security Labs, 2018. [Online]. Available: https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/

[9] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.

[10] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: a logic-based network security analyzer," in *Proc. 14th USENIX Security Symp.*, 2005.

[11] X. Ou, W. F. Boyer, and M. A. McQueen, "A scalable approach to attack graph generation," in *Proc. 13th ACM Conf. Computer and Communications Security (CCS)*, 2006, pp. 336–345.

[12] J. Backes et al., "Semantic-based automated reasoning for AWS access policies using SMT," in *Proc. Formal Methods in Computer Aided Design (FMCAD)*, 2018.

[13] Amazon Web Services, “SimulatePrincipalPolicy,” AWS Identity and Access Management API Reference. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/APIReference/API_SimulatePrincipalPolicy.html
[14] Amazon Web Services, “Policy evaluation logic,” AWS IAM User Guide. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
[15] Amazon Web Services, “Service control policies (SCPs),” AWS Organizations User Guide. [Online]. Available: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html
[16] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.
[17] M. G. Kendall, “A new measure of rank correlation,” *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.
[18] CloudChain, GitHub repository. [Online]. Available: https://github.com/threetheodrummer/CloudChain